Getting the Register onto Your iPad — Step by Step

For: Michael Lyman, Greenway Marijuana **Slice 9 — merged as PR #1053, commit** `a713b00d`

Read this part first (2 minutes)

You said you know nothing about this, so I am going to assume exactly that and explain everything. There is no step below that expects you to already know something. If a step says "tap the blue button that says X," then there is a blue button that says X. Where Apple's own wording is confusing, I say so.

Here is the honest shape of what is ahead. Turning a website into an iPad app is not one big scary step — it is a lot of small, boring, clerical steps, and the only reason it feels hard the first time is that Apple's tools use words nobody explains. The good news is that **you only do the painful setup once**. After the first time, putting a new version on the iPad takes about ninety seconds.

What I did in this slice, before you touch anything. I found and fixed three settings that would each have wasted an evening of your time. You did not know about any of them and had no way to. They matter because none of them cause an error message on my end — the code compiles perfectly, every test passes, and then the app fails on the actual iPad. That is the worst kind of bug because it burns *your* time, not mine.

1. **The app claimed it needed a 32-bit processor.** This is a le—ıʏ Hn

newer, so yours is supported with room to spare. Your newer store iPads will work too. You do not need to update anything.

Part 1 — Set up the Mac (once, about 45 minutes)

Step 1 — Install Xcode

Xcode is Apple's app-building program. It is large — around 10 GB — so start this first and read ahead while it downloads.

1. On the Mac, click the **blue "A" icon** in the dock (the App Store).
2. In the search box at the top left, type `Xcode` and press **Return**.
3. Find **Xcode** by Apple. Click **GET**, then **INSTALL**.
4. Enter your Apple ID password or use Touch ID.
5. **Wait.** On good internet this is 30–60 minutes. Leave it alone; you can keep reading.

When it finishes, the button says **OPEN**.

Step 2 — Open Xcode once and let it finish

1. Open Xcode (App Store → **OPEN**, or Launchpad → Xcode).
2. It shows a licence agreement. Read or scroll, then click **Agree**.
3. It says **"Installing components..."** and asks for your Mac password. Type it and click **OK**.
4. Wait for it to finish, then you will see a **Welcome to Xcode** window.

Leave Xcode open.

Step 3 — Install the command line tools

Xcode has a second, smaller piece that the build needs.

1. In the very top menu bar, click **Xcode → Settings...** (older versions: **Preferences...**).
2. Click the **Locations** tab at the top of the window that opens.
3. Look at the **Command Line Tools** dropdown at the bottom.
4. If it is empty, click it and choose the Xcode version listed (something like `Xcode 26.x`). If it already shows a version, you are done.
5. Close the Settings window.

Step 4 — Get the code onto the Mac

Now we open the Terminal. Terminal is a window where you type commands instead of clicking. It looks intimidating and is not; you will paste in text and press Return.

1. Press **Command (⌘) + Space** together. A search bar appears in the middle of the screen.
2. Type `terminal` and press **Return**. A window with plain text opens.
3. Copy the line below, paste it into Terminal (**⌘V**), press **Return**:

```
git clone https://github.com/mblyman89/GREENWAY-WEBSITE.git ~/greenway
```

It prints several lines and may ask for your GitHub username and a password/token. When it stops and shows a fresh prompt, it is done.

> **If you already cloned this before,** do not run the line above. Instead run these two, one at a time: `cd ~/greenway` `git pull`

4. Now move into the folder and install the parts. Paste each line separately, pressing **Return** and waiting for the prompt to come back:

```
cd ~/greenway
```

```
npm install
```

`npm install` takes several minutes and prints a wall of text. Warnings in yellow are normal and fine. Only red text containing the word `ERR!` is a real problem.

> **If Terminal says `command not found: npm`,** Node is not installed. Go to https://nodejs.org, download the button marked **LTS**, open the downloaded file, click through the installer, then **close Terminal completely and open a new one** before retrying. The new window is important — an old window will not see the new install.

Part 2 — Tell Apple who you are (once, about 10 minutes)

Step 5 — Sign in to Xcode with your developer account

1. Switch to Xcode. In the top menu: **Xcode → Settings...**.
2. Click the **Accounts** tab.
3. Click the **+** button in the bottom-left corner of that window.
4. Choose **Apple ID**, click **Continue**.
5. Enter the Apple ID and password for your **approved developer account** (the one approved recently — not a personal Apple ID, if they differ).
6. Complete two-factor authentication if prompted.
7. Your account now appears in the left list. Click it once and confirm the right-hand side shows a team with the role **Account Holder** or **Admin**.
8. Close Settings.

The address question — please read before Step 6

Step 6 asks for your server address, and I have to flag something rather than guess, because guessing here would send your registers to the wrong place.

You told me your website is `www.greenwaymarijuana.com`. I checked all three possibilities directly, and here is exactly what each one does today:

Address	What actually happens
`https://www.greenwaymarijuana.com`	**Fails to connect.** The security certificate does not cover the `www.` version. A browser shows a scary warning; the app would simply fail.
`https://greenwaymarijuana.com`	Loads, but it is your **old WordPress site**. Asking it for register data returns a WordPress "page not found."
`https://greenwaywebsite1.vercel.app`	**This is the live back office.** I asked it for the register version and it answered correctly, matching the current code.

So the address that works right now is the `vercel.app` one. But I am deliberately **not** hard-coding that into the app, for two reasons. First, I do not want to bake a temporary-looking address into your registers without your say-so. Second, and more importantly, **the `www.` failure is a real problem worth fixing on its own** — customers typing `www.greenwaymarijuana.com` today get a security warning, which is bad for a legitimate business regardless of the register.

For today, use `https://greenwaywebsite1.vercel.app`. It works, it is secure, and you can change it later by rebuilding — it is one line and about ninety seconds. Nothing is locked in.

I have put the domain cleanup in the questions at the end. It is a genuine issue and I would like to fix it properly in its own slice.

Part 3 — Build the app (about 15 minutes)

Step 6 — Build the register bundle

Back in Terminal. Paste these one at a time:

```
cd ~/greenway
```

```
REGISTER_API_BASE="https://greenwaywebsite1.vercel.app" npm run register:build:ios
```

That second line looks odd, so here is what it means: everything before `npm` is you handing the build one piece of information — your server address — and everything after is the build command. They go on one line together.

What you should see first:

```
Greenway register — iPad build preflight
──────────────────────────────────────────

  OK   server address: The app will call https://greenwaywebsite1.vercel.app/api/pos/… for every request.
  OK   device requirements: [arm64]
  OK   export compliance: declared, no non-exempt encryption

All preflight checks passed. Building the iPad bundle…
```

That is the safety check I built in this slice, confirming all three fixes are live. Then Vite builds the bundle and Capacitor copies it into the iPad project. Finished when the prompt returns with no red `ERR!`.

> **If it says `Build stopped`,** read the numbered explanation it prints — it is written in plain English and tells you exactly what to fix. The most common cause is a typo in the address, or forgetting `https://` at the front.
>
> **If it says `command not found: cap`,** run `npm install` again in `~/greenway` and retry.

Step 7 — Open the project in Xcode

```
open ~/greenway/ios/App/App.xcworkspace
```

Important: it must be `App.xcworkspace`, not `App.xcodeproj`. They look nearly identical in Finder. The command above picks the right one, which is why I am giving you the command instead of asking you to double-click.

Xcode opens and may say **"Loading…"** or show a progress bar at the top while it fetches its packages. Wait until that finishes.

Part 4 — Prepare the iPad (once, about 10 minutes)

Step 8 — Plug in the iPad and trust the Mac

1. Connect the iPad to the MacBook with the Lightning cable.
2. The iPad shows **"Trust This Computer?"** — tap **Trust**.
3. The iPad asks for its passcode. Enter it.
4. If the Mac shows a Finder window for the iPad, you can close it.

Step 9 — Turn on Developer Mode on the iPad

Apple requires this before an iPad will run an app that did not come from the App Store. **This option does not exist until you have plugged the iPad into a Mac running Xcode at least once**, which you just did in Step 8. If you go looking for it beforehand you will not find it and will think something is broken.

On the **iPad**:

1. Open **Settings** (grey gear icon).
2. Tap **Privacy & Security** in the left-hand list.
3. Scroll to the **bottom** of the right-hand panel.
4. Tap **Developer Mode**.
5. Turn the switch **ON**.
6. It warns you and offers **Restart**. Tap **Restart**.
7. After it restarts, **unlock the iPad**.
8. A prompt asks **"Turn on Developer Mode?"** — tap **Turn On** and enter your passcode.

> **If you cannot find Developer Mode:** unplug the iPad, make sure Xcode is open on the Mac, plug it back in, wait 30 seconds, then look again. It appears only after the iPad has seen a Mac with Xcode.

Part 5 — Put the app on the iPad (about 10 minutes)

Step 10 — Set the signing team

"Signing" is Apple's way of stamping the app with your identity so the iPad trusts it.

1. In Xcode, look at the **left sidebar**. At the very top is a blue icon labelled **App**. Click it once.
2. The main area fills with settings. Near the top, find the row of tabs: **General**, **Signing & Capabilities**, **Resource Tags**, **Info**, **Build Settings**…
3. Click **Signing & Capabilities**.
4. Tick the checkbox **Automatically manage signing** if it is not already ticked.
5. In the **Team** dropdown, choose your developer team (your business name or your name).
6. Wait a few seconds. Xcode creates a provisioning profile by itself.

You want to see: a **Signing Certificate** line with a real value, and no red error text.

> **If you see "Failed to register bundle identifier"** — this usually means the identifier is taken. Do not change it yourself; tell me and I will sort it, because `com.greenwaymarijuana.register` is permanent once uploaded and I do not want it changed by accident.
>
> **If you see a red error about the team,** go back to Step 5 and confirm the account is signed in.

Step 11 — Choose your iPad as the destination

1. At the **top centre** of the Xcode window is a bar showing the app name and a device name.
2. Click the **device name** part (right-hand side of that bar).
3. A dropdown lists simulators and, under a heading like **iOS Device**, your real iPad by name.
4. **Choose your real iPad**, not a simulator. A simulator is a fake iPad on the Mac screen; it cannot test your scanner, printer, or drawer.

> **If your iPad is not listed:** check it is unlocked, check you tapped **Trust**, check Developer Mode is on, and try a different Lightning cable. A charge-only cable is the most common cause.

Step 12 — Run it

1. Click the **▶ (Play)** button at the top left of Xcode.
2. The Mac may ask for your password to access the keychain — enter it and, if offered, click **Always Allow** so it stops asking.
3. Xcode shows **"Building…"** then **"Installing…"** then **"Running App on ."**
4. **Watch the iPad.** The app installs and launches by itself.

The first build takes a few minutes. Later ones are much faster.

Step 13 — Trust the developer on the iPad

The first time only, the iPad may refuse to open the app and say the developer is not trusted.

On the **iPad**:

1. **Settings** → **General**.
2. Tap **VPN & Device Management**.
3. Under **Developer App**, tap your developer account name.
4. Tap **Trust "..."**, then **Trust** again in the popup.
5. Return to the Home screen and tap the **Greenway Point of Transaction** icon.

Part 6 — Connect the register (about 5 minutes)

The app is on the iPad. Now it needs to know which register it is.

Step 14 — Create a device credential in the back office

On the **Mac**, in a normal web browser:

1. Go to `https://greenwaywebsite1.vercel.app` and sign in as owner/manager.
2. Navigate to **Register Activity** → **POS devices**.
3. Provision a new device and bind it to the register it will be (for example **Sales Register 1**).
4. The page shows a **device id** and a **device key**.

> **The key is shown once and never again.** This is deliberate — the server keeps only a scrambled copy, so nobody, including me, can read it back. If you lose it, you rotate the key and get a new one; nothing is broken, it is just an extra minute. **Copy both values somewhere safe before leaving the page.**

Step 15 — Enter them on the iPad

On the **iPad**, the app shows a screen titled **Register setup** with two boxes.

1. Tap the box under **Device id** and enter the device id.
2. Tap the box under **Device key** and enter the device key.
3. Tap **Verify & save**.

About typing the key: it is **case-sensitive** — capital and lowercase letters are different characters. I know this bites people, so the app already turns off iPad auto-capitalisation and autocorrect on both boxes specifically to stop iPadOS quietly changing your first letter. Even so, **pasting is safer than typing**. Email the values to yourself, then copy and paste on the iPad.

> **If it says "Device key rejected,"** the id and key do not match. Re-copy both — the most common cause is a missing character at the start or end of the paste.

When it accepts, you land on the PIN screen. **Slice 9 is done.**

Part 7 — Putting a new version on later

Once the above is done, updates are quick:

```
cd ~/greenway
```

```
git pull
```

```
REGISTER_API_BASE="https://greenwaywebsite1.vercel.app" npm run register:build:ios
```

Then in Xcode press ▶. About ninety seconds. You will not repeat Parts 1, 2, 4, or 6.

What is deliberately NOT in this slice

I want to be straight about scope, because you raised two things that matter a great deal and I have not fixed them yet **on purpose**.

Your scanner will still be slow today. The 10-second driver's-licence read will still be 10 seconds after this slice. I know why, and the diagnosis was already in your repo from earlier work: your **Socket Mobile DuraScan D760** is in "Basic Mode," where it pretends to be a keyboard and types the licence out one character at a time. A driver's licence barcode holds roughly 300 to 1,100 characters. At keyboard speed, that is your ten seconds.

The fix is real and I have confirmed it is available: the D760 also has an **Application Mode** that hands the whole barcode over in one chunk, and it is MFi-certified for exactly this. The earlier research concluded this was impossible from Safari and noted it would only become possible "if a native iOS app ships." **This slice is that app shipping.** So the thing that was blocked is now unblocked — but it needs Socket Mobile's SDK and a registration step with them, which is its own slice with its own testing. Doing it in the same slice as "make the app install" would mean that if anything failed, we would not know which change caused it.

Receipt printing is not wired up yet. You were clear that the budtender must never be bounced out of the app to print, and you are right to insist — the current web approach does exactly that, and it is not acceptable for a real counter. There is a proper enterprise solution: printing and the cash-drawer kick both happen inside the app with no app switch. I have verified the drawer can be opened in-app.

I have not built it because **I still do not know what printer is on your front counter**, and this is the one thing I genuinely cannot determine from the repo. Your earlier notes recorded a decision to keep the store's existing Bluetooth receipt printer and flagged "owner to provide the exact make/model" — that action is still open. The TSP143IV in your equipment hub is the *online-order* printer on Ethernet, a different job. I will not guess at your counter printer, because guessing means picking the wrong SDK and wasting a slice.

Planned order: Slice 10 = the scanner (unblocked, no purchase needed). Slice 11 = printer and cash drawer (needs the model below).

Two questions

1. The web address. Confirm I should keep pointing the registers at `https://greenwaywebsite1.vercel.app` for now. Separately — and I think this is worth fixing regardless of the register — `www.greenwaymarijuana.com` currently **fails with a certificate error**, so any customer typing "www" gets a security warning. If you tell me where the domain is registered, I will lay out exactly how to fix that too.

2. The front-counter receipt printer. I need the **exact make and model** of the Bluetooth receipt printer at your counter — the one that kicks the cash drawer. Easiest way: look at the **label on the bottom or back** of the printer and send me a photo of it. Any model number will do. If it is a Star or an Epson, say so; those are the two most likely and each needs a different SDK. Without this I cannot start Slice 11.

For the record — what was verified, not assumed

Per your standing rule that we build from fact and never from memory:

- Merged as PR #1053, commit `a713b00d`, fast-forward onto `main`.
- `tsc` 0 errors; `eslint` 0 problems.
- Full test suite: **522 files / 13,274 tests pass** (previous baseline 521 / 13,261). No regressions.
- `Info.plist` re-parsed after editing: valid, 19 keys, requirement is `['arm64']`, encryption answer is `False`.

- The safety check was **deliberately broken to prove it works**: putting `armv7` back failed 2 tests; deleting the encryption answer failed 1. A tripwire that cannot trip is not a tripwire.
- The build refusal was tested three ways: no address → refused; insecure `http://` → refused; correct `https://` → allowed.
- All three web addresses in "The address question" were tested live, not recalled.
- Your iPad was identified from model **ML3K2LL/A** as an iPad Pro 12.9-inch 1st generation (2015), and iPadOS 16.7.16 was **checked against Apple's current support table** rather than assumed.

One limitation I will state plainly rather than paper over: **I could not run the Capacitor sync step in my own environment**, because it requires a newer Node than my sandbox has. Everything either side of it is verified as described. That step is the ordinary `cap sync ios` that runs automatically as part of Step 6 on your Mac, and if it does anything unexpected, send me the text and I will read it.